```
            } else {
                app.showNotification('No data was selected.
Please select some data in document');
            }
        }
        WordCloud(document.getElementById('word-cloud-
placeholder'), {
            list: list,
            fontFamily: 'Finger Paint, cursive, sans-serif',
gridSize: 16,
weightFactor: 2,
            color: '#f0f0c0',
backgroundColor: '#001f00',
            shuffle: false,
rotateRatio: 0
        });
    }
```

9. Now, we will use the *Office.js* function to get the selected text from the document and pass it to the *draw()* method, on clicking the button (with the ID *cloud-from-selection*).

```
$("#cloud-from-selection").on('click', function () {
Office.context.document.getSelectedDataAsync(Office.
CoercionType.Text,
        function (result) {
                if (result.status === Office.
AsyncResultStatus.Succeeded) {
                    if (result.value == "") {
                        app.showNotification('No data
was selected. Please select some data in document');
                    } else {
                        draw(result.value);
                    }
                } else {
                    app.showNotification('Error:',
result.error.message);
                }
            }
        );
    });
```

10. Then, we will need a function to insert the word cloud image into the document. There are two ways of achieving this. We will have to check if the running version of Word supports the insertion of base64 encoded images, in which case we will directly insert it in the document, or else we will send the image to a server side script, save the image on the server and then insert an <*img*> tag with the URL of the saved image. The functions to do this will be as shown below:

```
function setHTMLImage(imageHTML) {
        Office.context.document.setSelectedDataAsync(
imageHTML, { coercionType: Office.CoercionType.Html },
            function (asyncResult) {
                if (asyncResult.status == "failed") {
                    app.showNotification('Error: ' +
asyncResult.error.message);
                }
            });
    }
    function setBase64Image(imageBase64Data) {
        Office.context.document.setSelectedDataAsync(imageBas
e64Data, {
            coercionType: Office.CoercionType.Image,
        },
        function (asyncResult) {
            if (asyncResult.status === Office.
AsyncResultStatus.Failed) {
                app.showNotification ("Action failed with
error: " + asyncResult.error.message);
            }
        });
    }
```

Finally, we will call these functions on clicking the button (with the ID *insert-cloud-to-document*).We will use some feature detection to call the appropriate function, as follows:

```
if (Office.context.requirements.
isSetSupported('ImageCoercion', '1.1')) {
var $canvas = $('#word-cloud-placeholder');
var imageData = $canvas[0].toDataURL();
imageBase64Data = imageBase64Data.replace(/^data:image\/
(png|jpg);base64,/, "");setBase64Image(imageData);
} else {
// get the imageURL from a server side script that saves the
image on a server and return the //URL.
var imageHTML = "<img "+"src='" + imageURL + "' img/>";
setHTMLImage(imageHTML)
}
```

11. That's pretty much it. Now you can hit the *Start* button in Visual Studio and see the application running, and if you want to download the complete source code, it is available on GitHub (*https://github.com/shivasaxena/Word-Cloud-Generator*). The final add-in looks like what's shown in Figure 2. END

By: Shiva Saxena

The author is a FOSS enthusiast. He is currently working as a consultant involved in developing Enterprise Application and Software as a Service (SaaS) products. He has hands on development experience with technologies like Android, Apache Camel, C#.NET, Hadoop, HTML5, Java, OData, Office, PHP and React, among others, and loves to explore new and bleeding edge technologies. He can be reached at *shivasaxena@outlook.com*